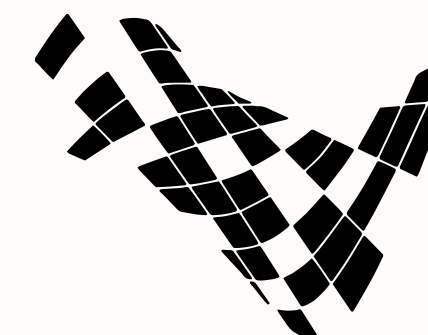


GAUSSIAN PROCESSES AS EMULATORS

Tim Rogers

June 2026

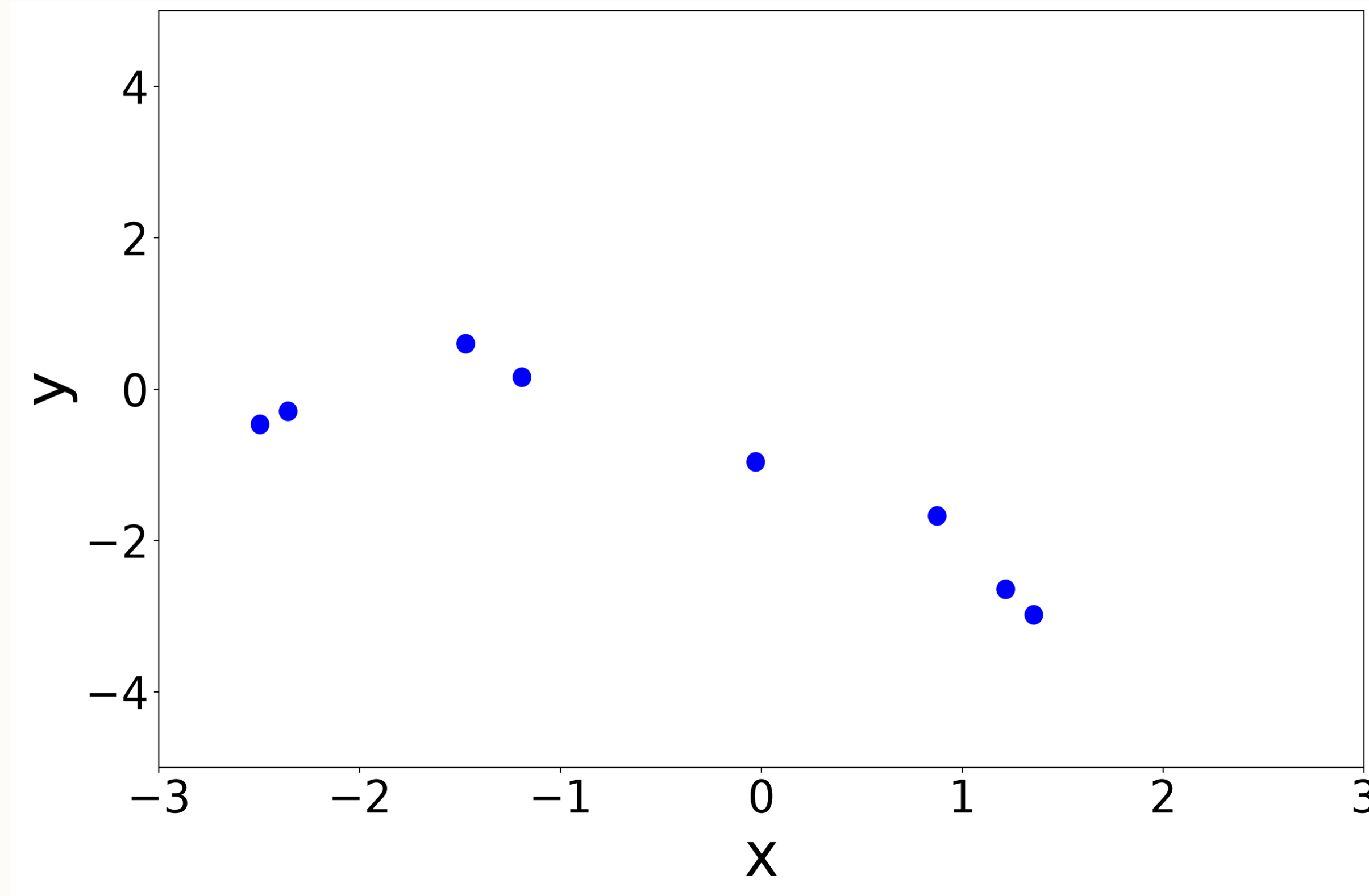


INTRO

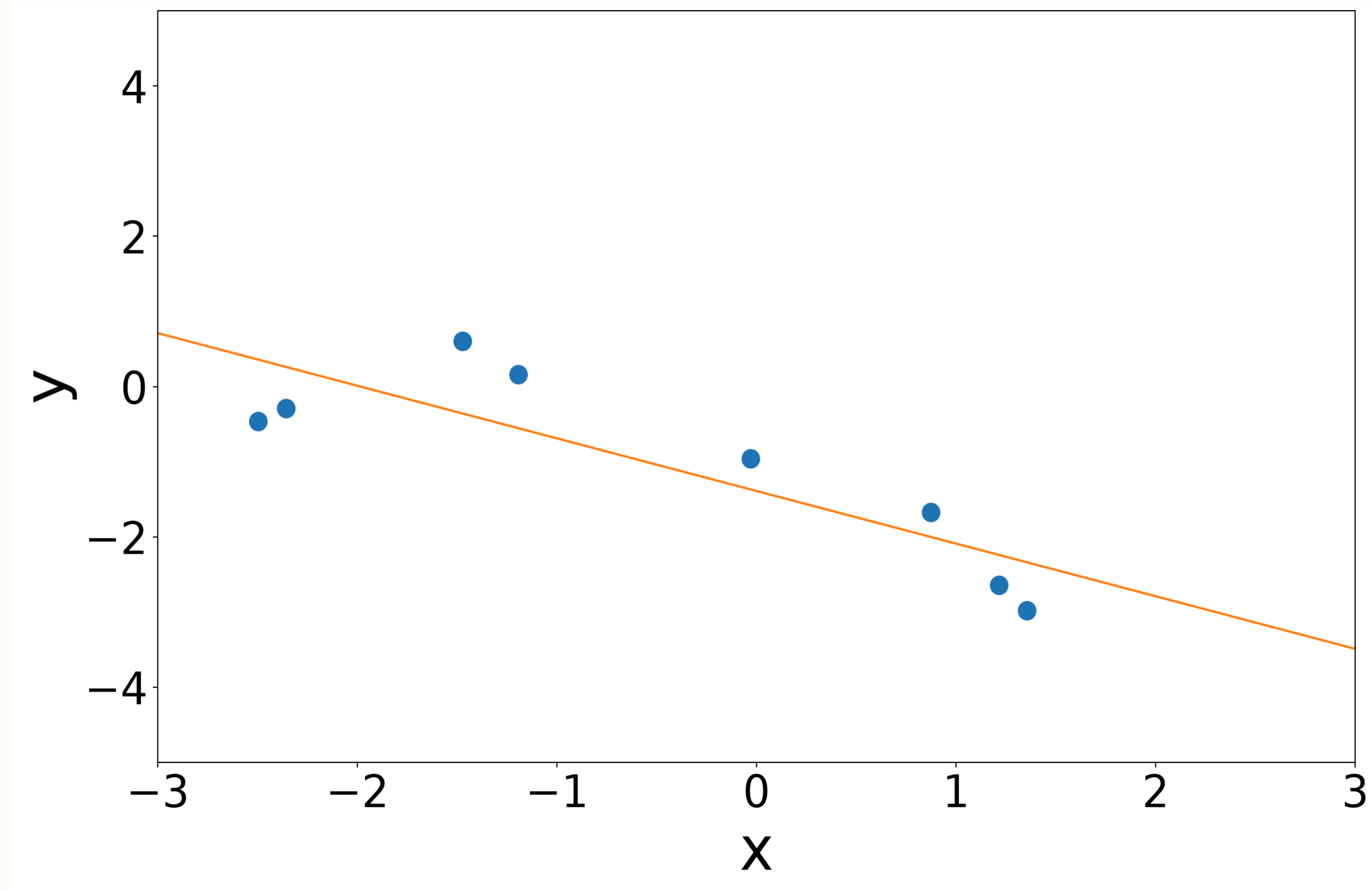
- Why do we build surrogates/emulators?
- Problems you might have with emulators?
 - You don't know if it matches the simulator
 - You don't know how much data you need to fit it
 - You **know** it won't be perfect but how bad is it?
- In the end emulation is just a special use of regression (**personal viewpoint**)



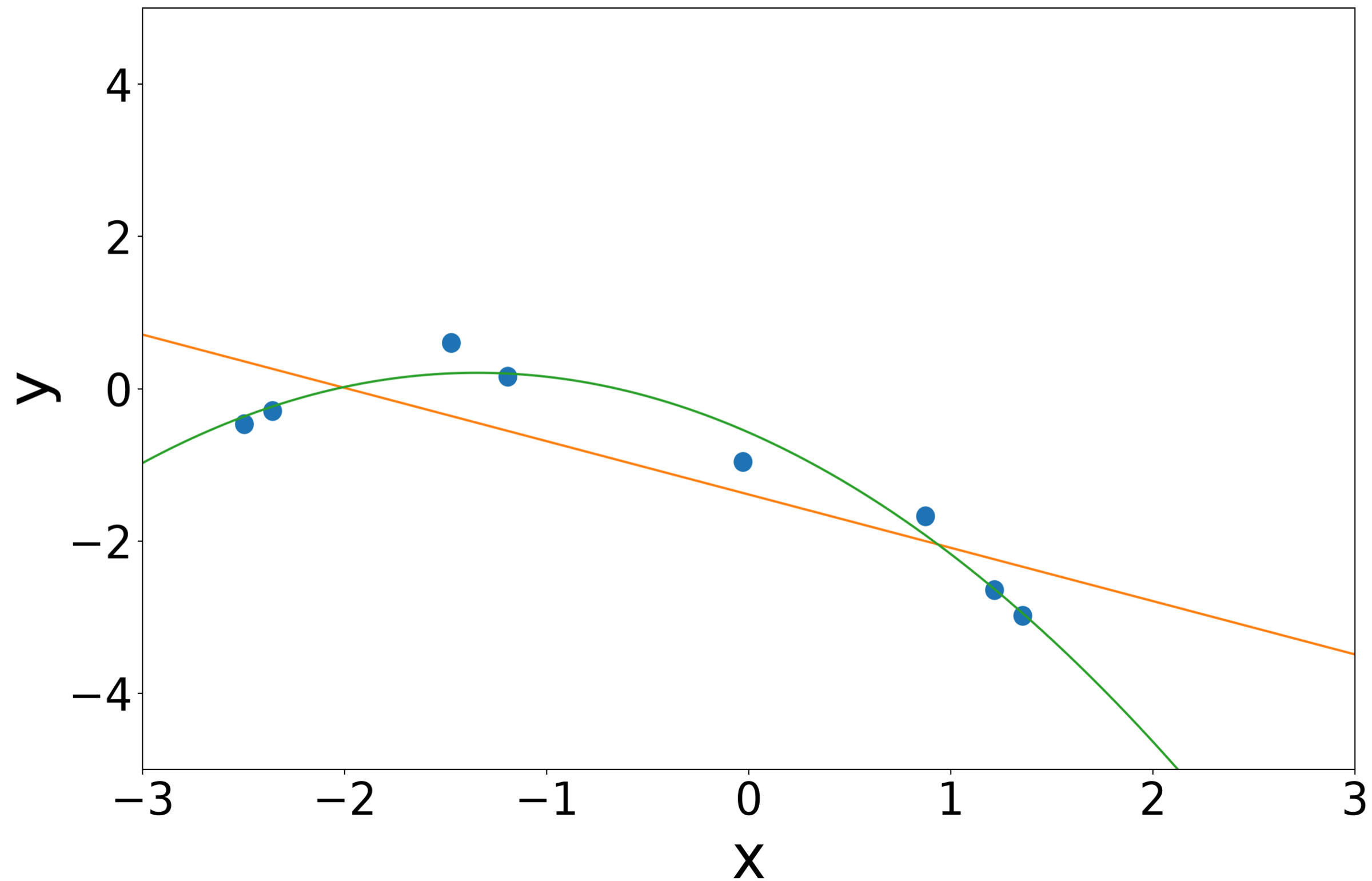
FITTING DATA



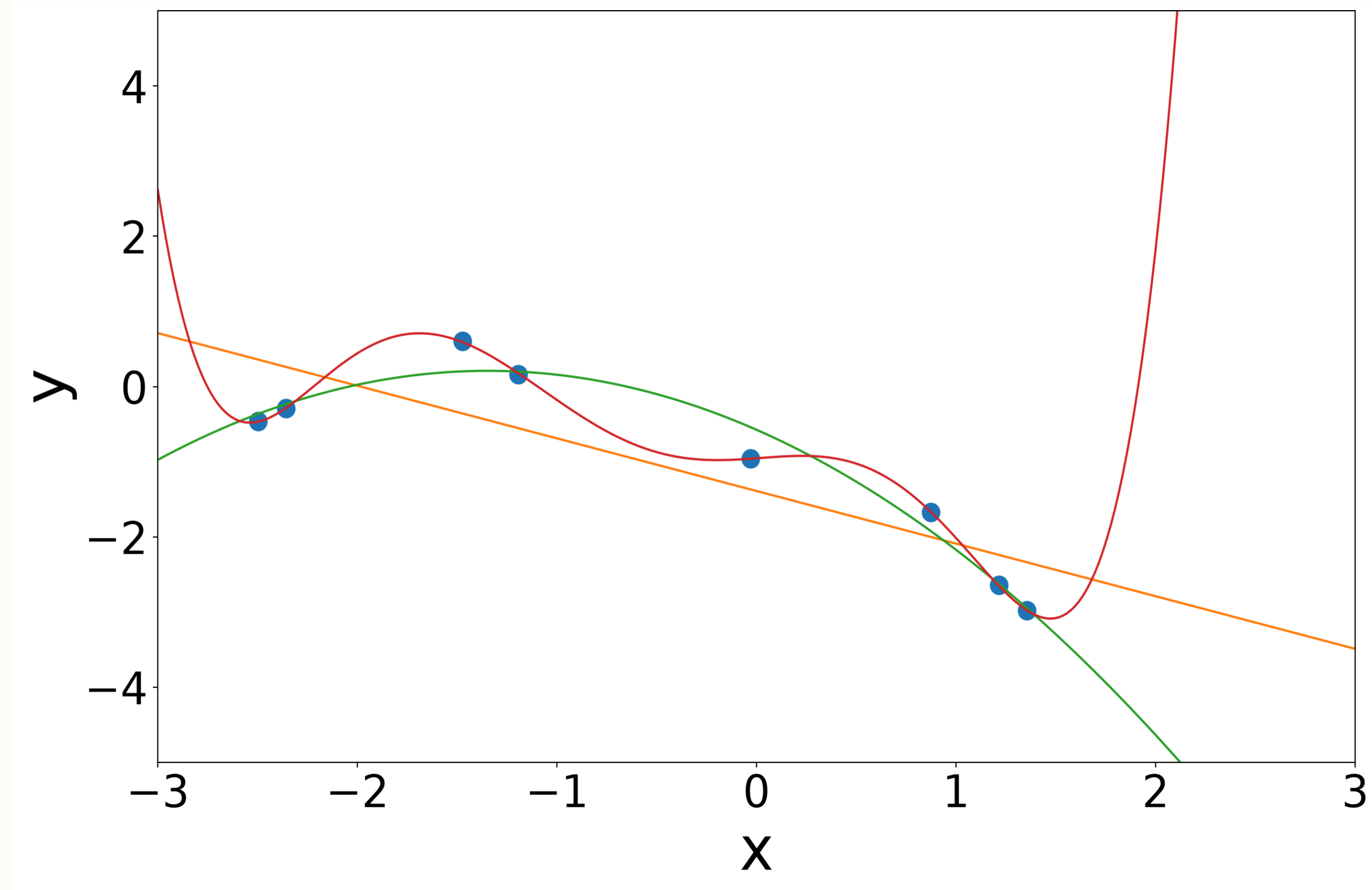
FITTING DATA



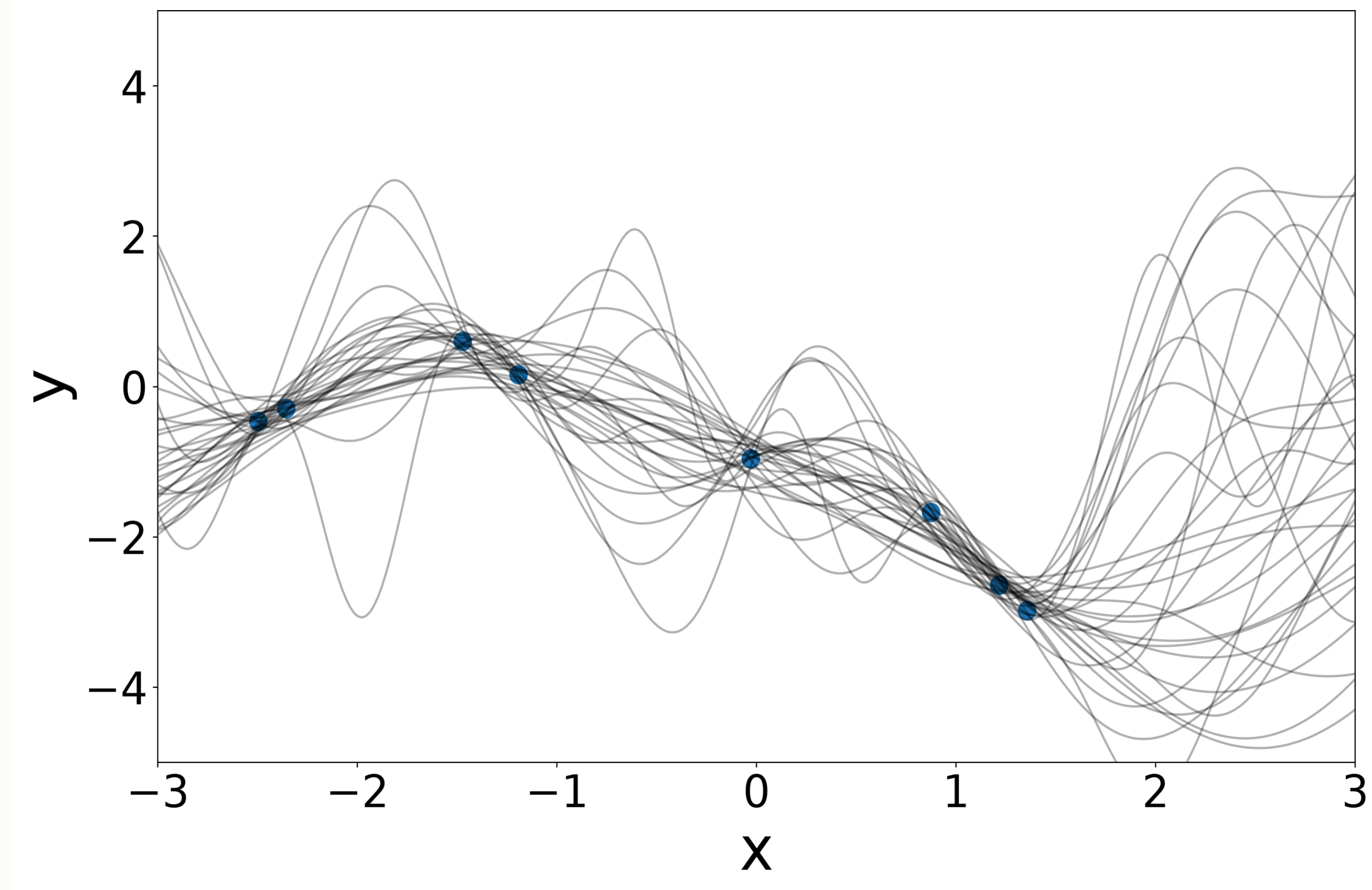
FITTING DATA



FITTING DATA



FITTING DATA



WHAT WE WANT

- Can we avoid needing to pick a basis?
- Can we find a way to cover the possibility of all these functions?
- Can we know the width of possibilities at any point in space?
- Is there a principled way to avoid needing to fit an infinite number of functions?



THE GAUSSIAN PROCESS

Gaussian Process

A probability distribution where every sample is a function, fully defined by a **mean function** $m(x)$ and **covariance function** or *kernel* $k(x, x')$.

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

This lets us define a way to learn a whole *distribution* of functions rather than just a single “line-of-best-fit”.



A GP ON TRAINING DATA

Another definition of the GP is that it is a multivariate Gaussian over infinite dimensions.

Since any finite subset of a multivariate Gaussian is marginally also a multivariate Gaussian, we only need to work with these.

If I have training data $\mathcal{D} = \{x_i, y_i\}_{i=1}^N$ and $y_i = f(x_i) + e_i$ with $e_i \sim \mathcal{N}(0, \sigma_n^2)$ then

$$\mathbf{y} = \mathcal{N} \left(m(\mathbf{x}), K_{ff} + \sigma_n^2 \mathbb{I} \right)$$

N.B. $[K_{ff}]_{ij} = k(x_i, x_j)$



MAKING PREDICTIONS

Our intuition will be that for inputs which are *similar* to those seen in “training” we should see outputs which are also *similar*.

To make predictions of $\mathbf{y}^*$ at new inputs $\mathbf{x}^* = \{x_j^*\}_{j=1}^{N^*}$ we will look at the extended **joint** distribution between $\mathbf{y}$ and $\mathbf{y}^*$.

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{y}^* \end{bmatrix} = \mathcal{N} \left(\begin{bmatrix} m(\mathbf{x}) \\ m(\mathbf{x}^*) \end{bmatrix}, \begin{bmatrix} K_{ff} + \sigma_n^2 \mathbb{I}_N & K_{ff^*} \\ K_{f^*f} & K_{f^*f^*} + \sigma_n^2 \mathbb{I}_{N^*} \end{bmatrix} \right)$$

Finally we can condition to get $p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}, \mathbf{x}^*)$

$$\mathbf{y}^* | \mathcal{D} \sim \mathcal{N} \left(m(\mathbf{x}^*) + K_{f^*f} \boldsymbol{\alpha}, K_{f^*f^*} - K_{f^*f} [K_{ff} + \sigma_n^2 \mathbb{I}]^{-1} K_{ff^*} + \sigma_n^2 \mathbb{I} \right)$$



BUILDING A GP EMULATOR

We now have everything (ish) to build an emulator:

1. Choose $m(x)$ and $k(x, x')$
2. Acquire $\mathcal{D}$ from a *small* number of simulator runs
3. Validate at hold out points $\mathcal{D}_v$
4. Predict across domain $x \in \Omega$ to get mean and covariance

Each of these has some challenges.



1. DEFINING THE GP

The Mean Function

This is a completely free choice of any parametric function of x , e.g. a linear or quadratic trend. Even better if this has *physical* inductive bias.

However, you will need to learn the parameters of this function. Easy if it is something linear ($H\Phi(x)$), harder if not.

We should also be aware that it has been shown that even if the parameters of that mean function are physically meaningful, their estimate may be biased.



1. DEFINING THE GP

The Covariance Function

The covariance function encodes our belief about the *family of functions* in this GP distribution.

We can make this simple, e.g. it's a straight line, or complex, e.g. it could be any possible polynomial. We can even use some physical knowledge here.

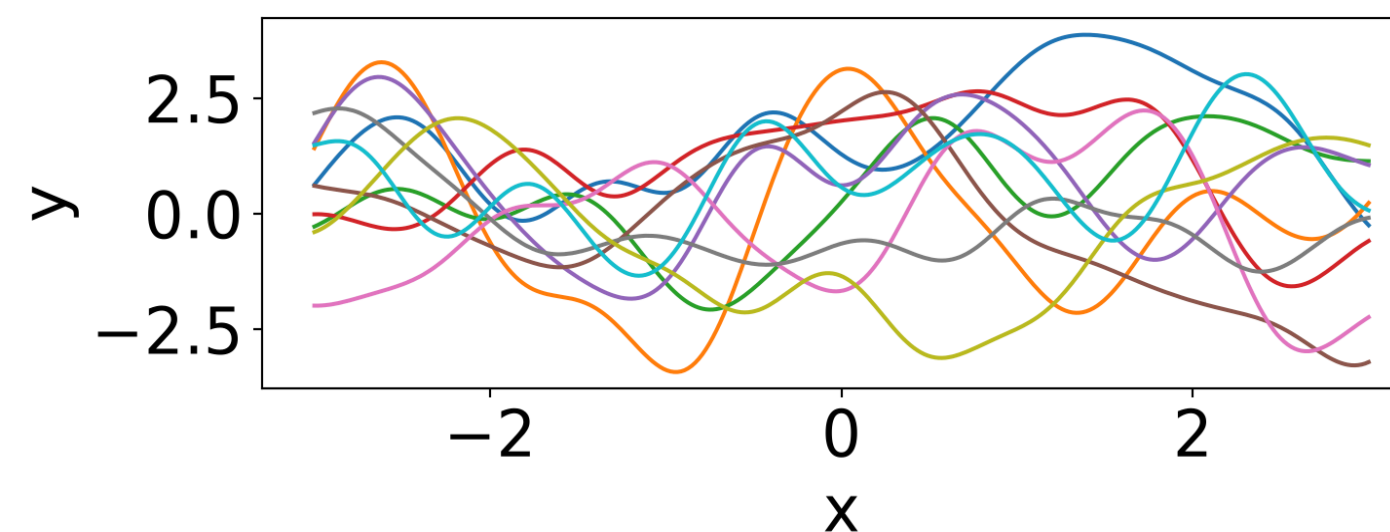
The covariance function typically has a (small) number of *hyperparameters* to optimise.



CHOICE OF COVARIANCE FUNCTION

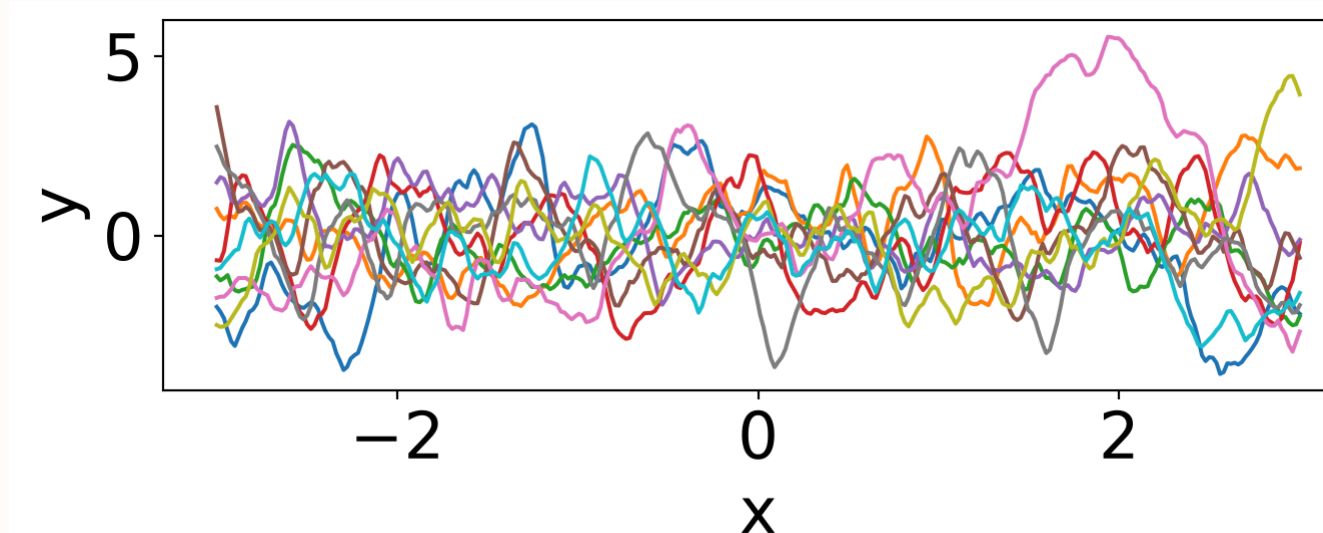
Squared Exponential:

$$k_{\text{SE}}(x, x') = \sigma_f^2 \exp \left\{ -\frac{\|x - x'\|_2^2}{2\ell^2} \right\}$$



Matérn 3/2

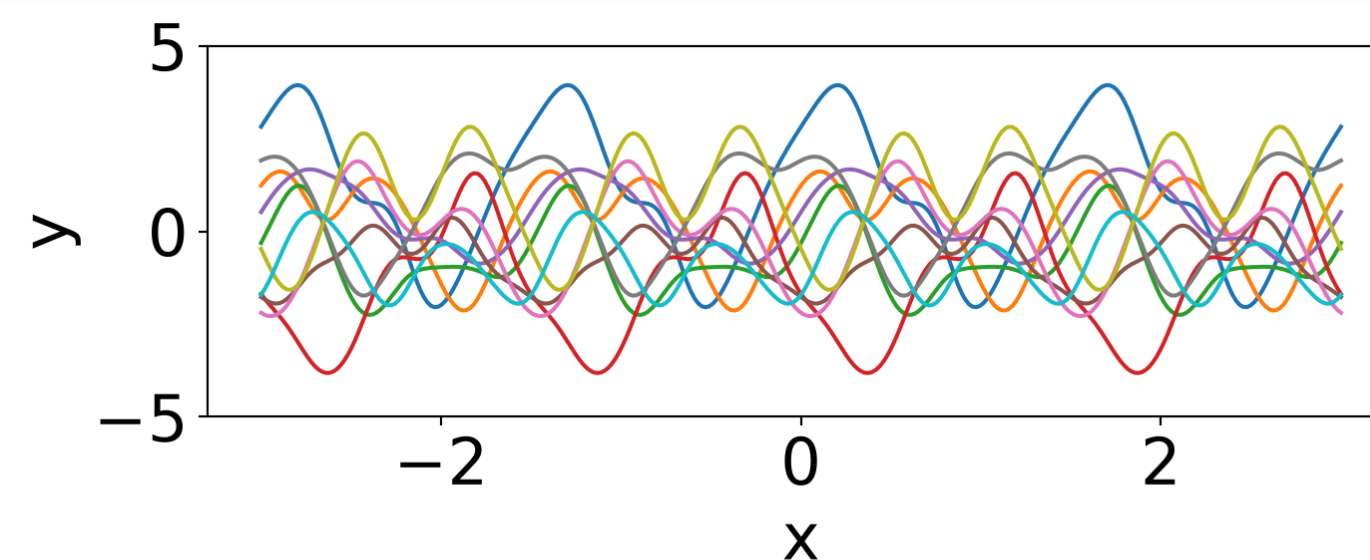
$$k_{\text{M32}}(x, x') = \sigma_f^2 \left(1 + \frac{\sqrt{3}d}{\ell} \right) \exp \left\{ -\frac{\sqrt{3}d}{\ell} \right\}$$



CHOICE OF COVARIANCE FUNCTION

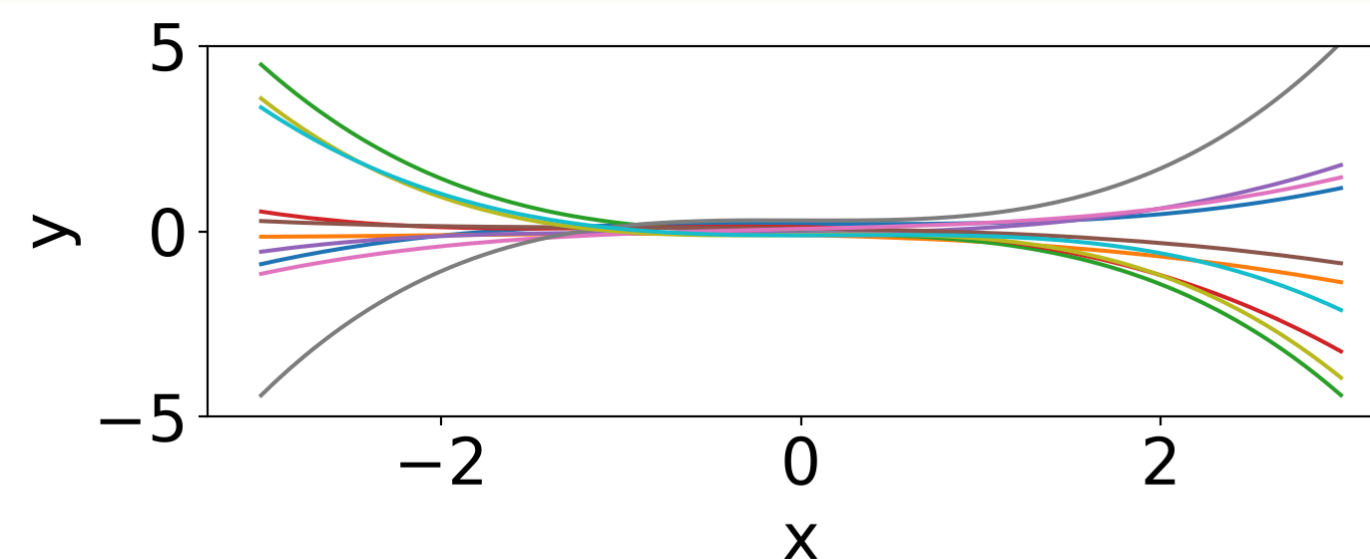
Periodic

$$k_{\text{Per}}(x, x') = \sigma^2 \exp \left(-\frac{2 \sin^2 \left(\frac{\pi |x - x'|}{p} \right)}{\ell^2} \right)$$



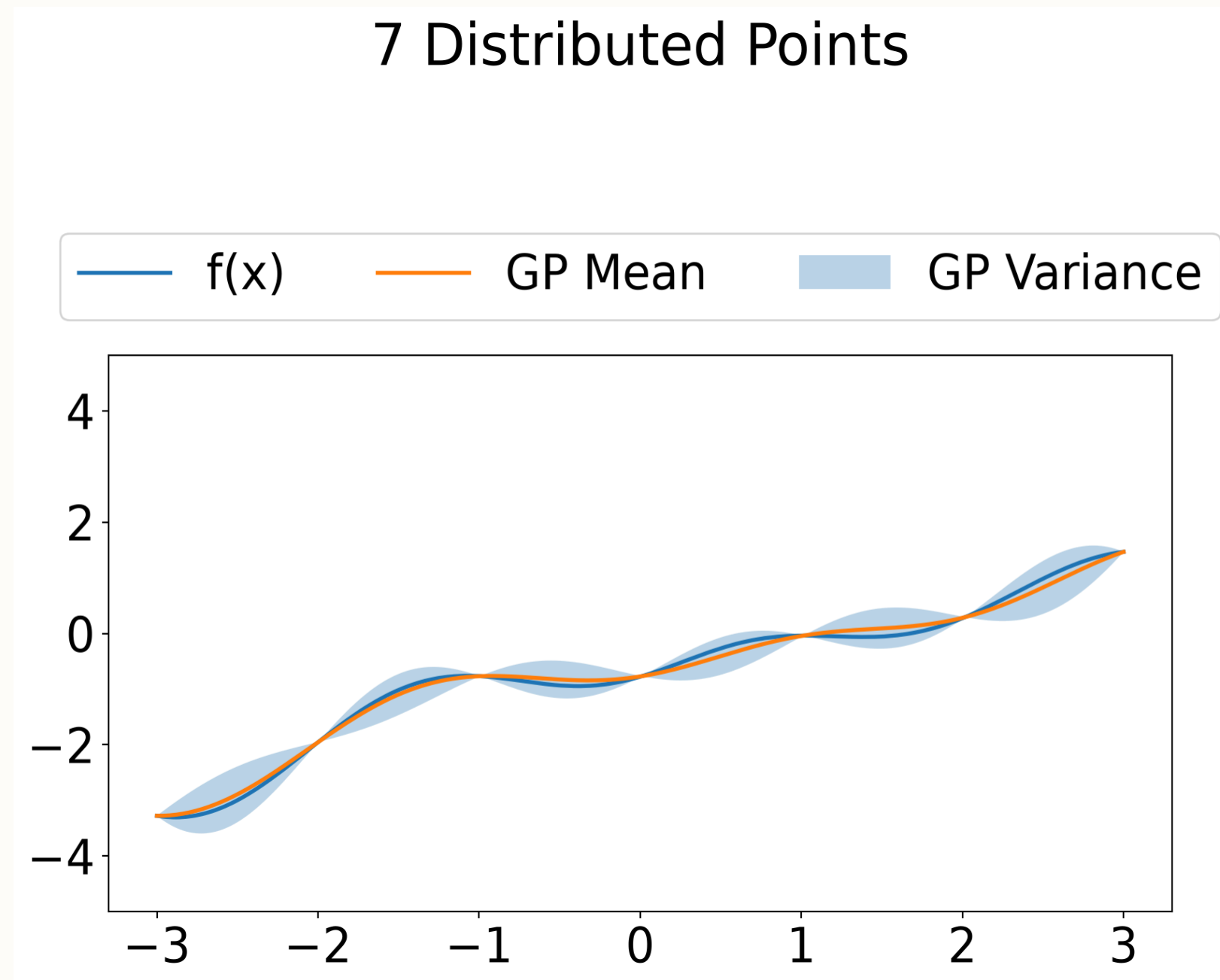
Linear Basis

$$k_{\text{LB}}(x, x') = \sigma_f^2 (\Phi(x) \cdot \Phi(x'))$$



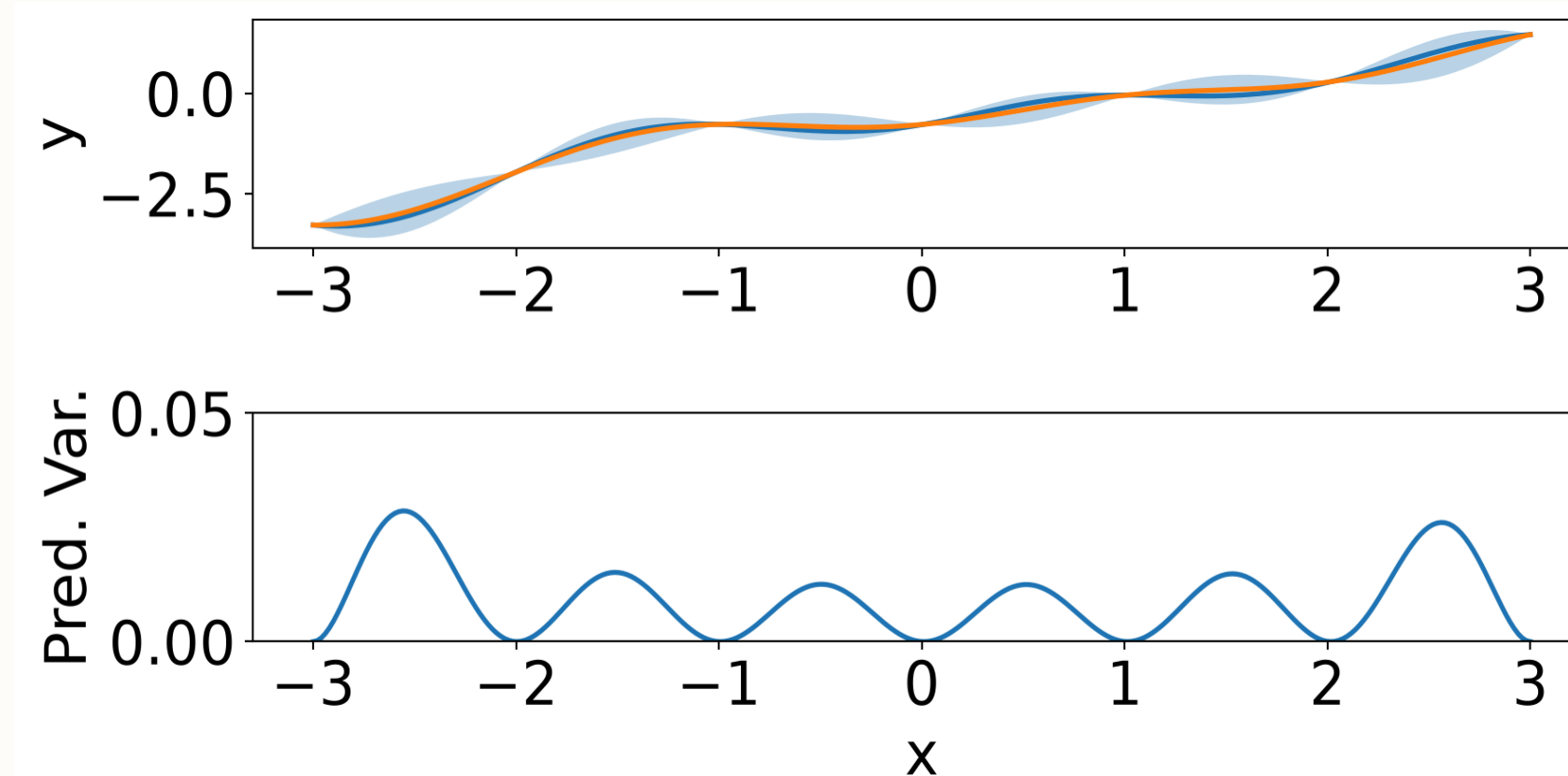
2. COLLECTING DATA

- As we are learning a model to represent the simulator we are necessarily dependent on where we initially run the simulator
- We want to make sure our data cover the space
- In higher dimensions this is more challenging, using Generalized Latin Hypercubes is an option



3. VALIDATION

- It is generally good practice to test your emulator against some held-out data.
- We can do the usual things, e.g. RMSE on some other points.
- We have the advantage of being able to evaluate uncertainty as well
- This is still available for unseen test points



4. PREDICTION

- As you have seen we can use the GP to predict at new inputs $\mathbf{x}^*$.
- We will talk about the *predictive distribution* from the GP, that is we get the probability $p(\mathbf{y}^*|\mathbf{x}^*, \mathcal{D})$
- We get a mean prediction $\mathbb{E} [\mathbf{y}^*]$ - the most likely values
- And the (co)variance of those quantities $\mathbb{V} [\mathbf{y}^*]$ - our degree of uncertainty
- We can then look at the marginal distributions or the joint over $\mathbf{y}^*$.

Warning

We can't guarantee that our predictions are accurate or our uncertainty is *correct*. Remember $\mathbb{V} [\mathbf{y}^*]$ is just a function of the similarity of $\mathbf{x}$ and $\mathbf{x}^*$.



WHAT ABOUT TRAINING?

Unlike a, e.g., neural network we don't have an extensive training phase because the model is *nonparametric*. However,

Hyperparameters

- Although we don't have growing numbers of parameters with model complexity
- Our kernels typically have some *hyperparameters* to optimise

Computational Cost

- Rather than distilling information from training data into the model, we explicitly condition on all observed data
- Inverting $[K_{ff} + \sigma_n^2 \mathbb{I}]$ has a cost of $\mathcal{O}(N^3)$



HYPERPARAMTER TUNING

This is one of the most delicate bits of fitting GPs and we have several options.

1. Choose some hyperparameters based on physical knowledge
2. Optimise for hyperparameters - use the log marginal likelihood

$$p(y|X; \phi) = \int p(y|f; \phi)p(f|X; \phi)df$$

3. Optimise for *maximum a posteriori* hyperparameters - include prior $p(\phi)$
4. Infer $p(\phi|\mathcal{D})$ then marginalise over possible hyperparameters

N.B. our simulators are often assumed noise free but be careful, see [Andrianakis and Challenor \(2012\)](#)



COMPUTATIONAL CONSIDERATIONS

Naïvely a GP will be $\mathcal{O}(N^3)$ to compute and this has been the subject of much research in the field.

In the emulator setting the computation time is still orders of magnitude less than the simulator, however, memory requirements become an issue.

If your dataset is large a variety of approaches exist:

- Inducing point low-rank approximations (FITC, PITC, etc.)
- Variational methods (e.g. VFE, VFF)
- Expectation propagation / Power EP
- Eigen-basis expansions
- SDE representations



WHY DOESN'T EVERYONE USE GPs

The following are still difficult to manage (in my opinion):

- High-dimensional outputs
- Very high-dimensional inputs
- Non-Gaussian likelihoods (partly)
- Discontinuous outputs
- Non-stationary functions



SUMMARY

- GPs give us a principled way of fitting functions and understanding uncertainty
- We avoid difficult training by being nonparametric but may still be sensitive to hyperparameters
- Computational cost can be higher (but is largely solved)
- Be careful about your modelling assumptions (general advice!)



GAUSSIAN PROCESSES AS EMULATORS

Tim Rogers

June 2026

